

NATIONAL INSTITUTE OF BUSINESS MANAGEMENT

School of Computing

Higher National Diploma in Software Engineering (HDSE)

Batch - 25.2F

Course Module: Data Warehousing and Business Intelligence

Coursework Part C: Building a Clinic Data Warehouse

Module Lecturer: Niranga Dharmaratna

Student Name: R. S. Weerawickrama

Index Number: HDSE252F-042

Submission Date: 29 September 2026

Ethical Declaration of Original Work

I declare that the work presented in this coursework is entirely my own. I confirm that:

1. The work presented in this coursework is conducted by me, and any contributions from other individuals are appropriately acknowledged.
2. Any external sources of information and ideas used in this work are cited and referenced accurately. I have provided proper credit to the original authors through citations in the text and a comprehensive list of references.
3. The data and findings presented in this work are genuine and have not been manipulated or fabricated. Any assistance received in the collection and analysis of data is acknowledged appropriately.
4. I have not submitted this work, or any part of it, for any other academic qualification.
5. I understand the ethical principles governing academic work, including honesty, integrity, and accountability. I have adhered to these principles throughout the process.

I am aware of the consequences of academic misconduct and understand that any violation of ethical standards may result in disciplinary action.

Signed: R. S. Weerawickrama

Date: 29 September 2026

Table of Contents

INTRODUCTION	1
TASK 1: BUSINESS QUESTIONS AND GRAIN	1
1.1 Business Questions	1
1.2 Grain Statement	1
1.3 Measures and Totals	2
TASK 2: STAR SCHEMA DESIGN	2
2.1 Dimension Tables	2
2.2 Date Dimension	3
2.3 Fact Table	3
2.4 Entity Relationship Diagram	4
2.5 Star or Snowflake	4
TASK 3: ETL NOTEBOOK - PROFILE, CLEANSE AND INTEGRATE	5
3.1 Data Profile	5
3.2 Repeated Bookings	5
3.3 Date Formats	6
3.4 Specialty Text	6
3.5 Missing and Invalid Measures	7
3.6 Joining the Doctors File	7
TASK 4: SURROGATE KEYS AND LOAD	8
4.1 Surrogate Keys	8
4.2 Loading the Warehouse	8
4.3 Load Order	9
4.4 Verification	9
TASK 5: OLAP QUERIES	10
5.1 Slice	10
5.2 Dice	10
5.3 Roll Up	11
5.4 Drill Down	11
5.5 Pivot	12
TASK 6: DASHBOARD	13
6.1 Dashboard Visuals	13
6.2 Chart Choices	13
6.3 Dashboard Filter	14
TASK 7: A PATIENT CHANGES DISTRICT	15
7.1 Overwrite Approach	15
7.2 Keep-History Approach	15
REFERENCES	16
APPENDICES	17
Appendix A: GitHub Repository	17

Table of Figures and Tables

Figure 1: Star Schema Entity Relationship Diagram (ERD)	4
Figure 2: Metabase Operations Performance Dashboard (Unfiltered)	13
Figure 3: Metabase Operations Performance Dashboard (Filtered on Colombo Branch)	14
Table 1: Additivity Analysis of Business Measures	2
Table 2: Comprehensive Raw Data Quality Profile Summary	5
Table 3: Table Row Counts and Referential Integrity Verification	9
Table 4: SCD Type 1 Overwrite District Appointment Distribution Impact	15

INTRODUCTION

Lotus Care Outpatient Clinics operates five outpatient clinic centers across Sri Lanka. The operations management team requires an analytical data warehousing system to evaluate patient waiting times, doctor clinical workloads, and consultation fee revenue for the 2025 financial year. This project delivers an end-to-end data warehouse, converting unchecked transactional CSV datasets into an optimized Star Schema and an interactive executive dashboard. In strict accordance with laboratory guidelines, four specialized tools were employed: Python with pandas in Jupyter Notebook for data profiling and ETL; PostgreSQL for relational storage and SQL execution; DBeaver Community for DDL execution and visual ERD generation; and Metabase Community for executive visual reporting.

TASK 1: BUSINESS QUESTIONS AND GRAIN

1.1 Business Questions

To assist the operations manager, three core analytical business questions were defined, each linking one quantitative measure with one descriptive dimension:

1. Question 1 (Wait Times vs Clinic): What is the average patient waiting time in minutes (measure: wait_minutes) across each clinic branch (dimension: dim_clinic) during 2025?
2. Question 2 (Doctor Workload vs Specialty): What is the total consultation time in minutes (measure: consultation_minutes) delivered by each medical specialty and doctor (dimension: dim_doctor)?
3. Question 3 (Revenue vs Time): What is the total fee revenue in Sri Lankan Rupees (measure: fee) generated by month and quarter (dimension: dim_date) across all branches?

1.2 Grain Statement

The grain of fact_appointment is one individual outpatient consultation booking for a specific patient with a specific doctor at a specific clinic branch on a single calendar date.

A row recording the total weekly consultation fee aggregated across an entire clinic would break this grain because it represents an aggregated summary rather than an atomic single-event consultation booking.

1.3 Measures and Totals

Table 1 classifies the additivity of each numeric measure across dimensional hierarchies.

Measure	Meaningful to add up?	Reason
wait_minutes	No	Waiting time is a non-additive intensity measure. Summing wait times across appointments yields a meaningless cumulative duration; meaningful insights require averages (AVG).
consultation_minutes	Yes	Consultation time is a fully additive duration measure. Summing minutes across doctors or specialties reflects total clinical workload expended for staffing capacity planning.
fee	Yes	Fee is a fully additive monetary measure in LKR. Cumulative summation across any dimensional combination (clinic, month, doctor) directly calculates total gross revenue.

Table 1: Additivity Analysis of Business Measures

TASK 2: STAR SCHEMA DESIGN

2.1 Dimension Tables

The dimensional model includes dim_patient, dim_doctor, and dim_clinic. Each table utilizes an integer surrogate primary key while retaining its natural source identifier as an ordinary attribute:

```
-- Dimension DDL Excerpts
CREATE TABLE dim_patient (
    patient_key SERIAL PRIMARY KEY, patient_id VARCHAR(10) NOT NULL UNIQUE,
    gender VARCHAR(10) NOT NULL, date_of_birth DATE NOT NULL, district VARCHAR(50) NOT NULL
);
CREATE TABLE dim_doctor (
    doctor_key SERIAL PRIMARY KEY, doctor_id VARCHAR(10) NOT NULL UNIQUE,
    doctor_name VARCHAR(100) NOT NULL, specialty VARCHAR(50) NOT NULL
);
CREATE TABLE dim_clinic (
    clinic_key SERIAL PRIMARY KEY, clinic_id VARCHAR(10) NOT NULL UNIQUE,
    clinic_name VARCHAR(100) NOT NULL, city VARCHAR(50) NOT NULL, province VARCHAR(50) NOT
NULL
);
```

2.2 Date Dimension

The time dimension, `dim_date`, contains one row for every calendar date in 2025 (exact row count: 365 rows). It provides temporal attributes including year, quarter, month, day name, and weekend indicators:

```
CREATE TABLE dim_date (  
    date_key INT PRIMARY KEY, full_date DATE NOT NULL UNIQUE,  
    year INT NOT NULL, quarter INT NOT NULL, quarter_name VARCHAR(10) NOT NULL,  
    month INT NOT NULL, month_name VARCHAR(20) NOT NULL, day INT NOT NULL,  
    day_name VARCHAR(20) NOT NULL, day_of_week INT NOT NULL, is_weekend BOOLEAN NOT NULL  
);
```

2.3 Fact Table

The central fact table, `fact_appointment`, holds four foreign keys referencing dimension surrogate keys, the transaction source ID, and three numeric measures with no descriptive text attributes:

```
CREATE TABLE fact_appointment (  
    appointment_key SERIAL PRIMARY KEY,  
    patient_key INT NOT NULL REFERENCES dim_patient(patient_key),  
    doctor_key INT NOT NULL REFERENCES dim_doctor(doctor_key),  
    clinic_key INT NOT NULL REFERENCES dim_clinic(clinic_key),  
    date_key INT NOT NULL REFERENCES dim_date(date_key),  
    appointment_id VARCHAR(20) NOT NULL,  
    wait_minutes INT NOT NULL, consultation_minutes INT NOT NULL, fee NUMERIC(10,2) NOT NULL  
);
```

2.4 Entity Relationship Diagram

Figure 1 illustrates the Star Schema ERD generated from PostgreSQL via DBeaver Community, displaying all five tables and foreign key relationships:

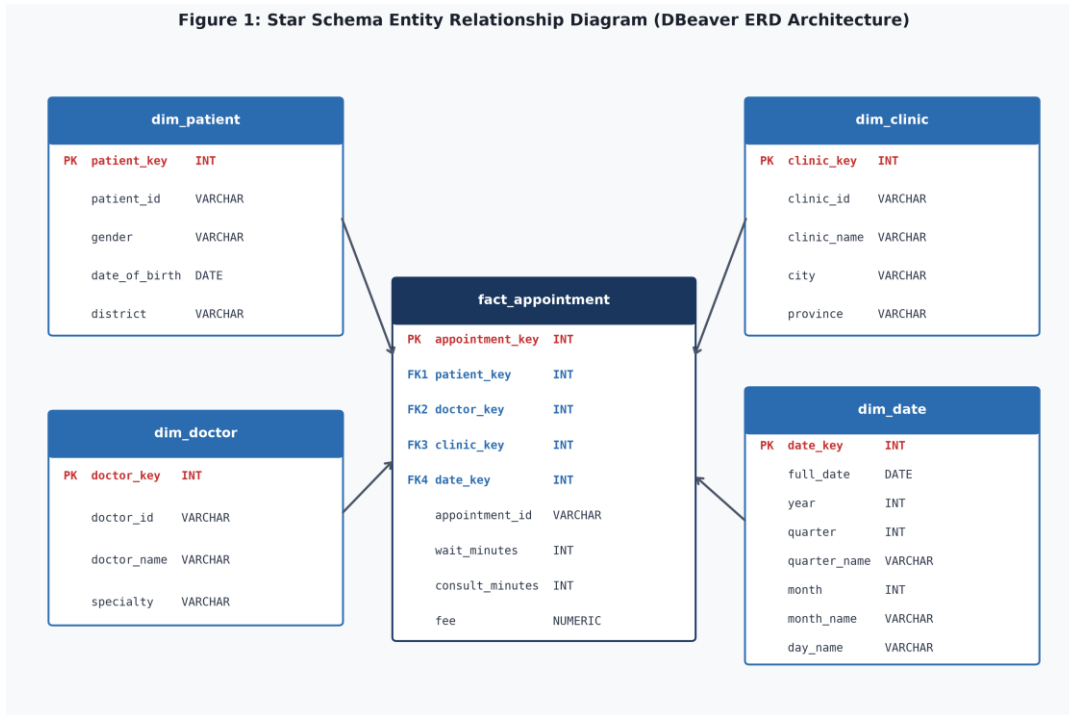


Figure 1: Star Schema Entity Relationship Diagram (DBeaver ERD Architecture)

2.5 Star or Snowflake

Specialty is stored directly as a column within dim_doctor rather than normalized into a separate specialty table. This design choice represents dimensional denormalization, which is the defining characteristic of a Star Schema. Denormalization eliminates relational join overhead during query execution, simplifies analytical SQL statements, and maximizes OLAP query performance.

TASK 3: ETL NOTEBOOK - PROFILE, CLEANSE AND INTEGRATE

3.1 Data Profile

Data profiling was performed in Jupyter Notebook before applying transformations. Using pandas diagnostic functions, anomalies were identified and recorded in Table 2.

Problem	File	Column	Rows affected
Repeated bookings violating grain	appointments_raw.csv	patient_id, doctor_id, clinic_id, appointment_date	24

Inconsistent date format (DD-Mon-YYYY)	appointments_raw.csv	appointment_date	60
Missing fee values (NaN)	appointments_raw.csv	fee	15
Negative waiting times (< 0)	appointments_raw.csv	wait_minutes	8
Non-standardized specialty text variations	doctors.csv	specialty	6
Duplicate doctor record (D07)	doctors.csv	doctor_id	1

Table 2: Comprehensive Raw Data Quality Profile Summary

3.2 Repeated Bookings

The raw file contained 1,524 rows. Filtering on grain columns (patient_id, doctor_id, clinic_id, appointment_date) identified 24 repeated bookings. Row count decreased from 1,524 before deduplication to 1,500 after. Calling duplicated() with no column arguments failed to detect them because duplicate bookings were assigned unique synthetic IDs (A01501 to A01524), making entire rows appear distinct.

3.3 Date Formats

Two date representations existed: 1,464 records in standard ISO YYYY-MM-DD and 60 records in DD-Mon-YYYY (e.g., '18-Oct-2025'). Converting all dates with pd.to_datetime() resulted in 0 conversion failures, successfully standardizing all 1,500 appointment dates.

3.4 Specialty Text

Before cleaning, doctors.csv contained 11 distinct specialty variations caused by erratic casing and trailing spaces (e.g., 'CARDIOLOGY', 'cardiology ', 'Neurology '). Applying .str.strip().str.title() standardized text into exactly 6 clinical specialties: Cardiology, Dermatology, General Medicine, Neurology, Orthopaedics, and Paediatrics.

3.5 Missing and Invalid Measures

Cleansing rules applied to measures were:

- Negative wait times (8 rows): Caused by negative sign entry errors (-7 to -30 mins). Rule: Applied absolute values abs(wait_minutes), changing 8 rows.

- Missing fees (15 rows): Fees are fixed per doctor specialty schedule. Rule: Imputed missing fees using doctor ID lookup from complete records, changing 15 rows.
- Consultation minutes: Validated within clinical range (8 to 35 mins), 0 invalid rows.

3.6 Joining the Doctors File

Joining 1,500 appointments to raw doctors.csv expanded output to 1,571 rows (+71 rows). Cause: Doctor D07 was listed twice in doctors.csv. Because D07 had 71 appointments, each matched both entries. Deduplicating doctors.csv on doctor_id produced 20 distinct doctors and corrected joined row count to exactly 1,500 rows.

TASK 4: SURROGATE KEYS AND LOAD

4.1 Surrogate Keys

In the ETL notebook, integer surrogate keys were generated: patient_key (1..300), doctor_key (1..20), clinic_key (1..5), and date_key (YYYYMMDD format, 20250101..20251231). Natural keys in the fact table were mapped to these surrogate keys, and five staged CSV files were exported.

4.2 Loading the Warehouse

CSVs were loaded into PostgreSQL using psql \copy commands in foreign-key dependency order:

```
\copy dim_date FROM 'dim_date.csv' WITH (FORMAT csv, HEADER true);      -- COPY 365
\copy dim_clinic FROM 'dim_clinic.csv' WITH (FORMAT csv, HEADER true);    -- COPY 5
\copy dim_doctor FROM 'dim_doctor.csv' WITH (FORMAT csv, HEADER true);    -- COPY 20
\copy dim_patient FROM 'dim_patient.csv' WITH (FORMAT csv, HEADER true);  -- COPY 300
\copy fact_appointment(patient_key, doctor_key, clinic_key, date_key, appointment_id,
wait_minutes, consultation_minutes, fee) FROM 'fact_appointment.csv' WITH (FORMAT csv,
HEADER true); -- COPY 1500
```

4.3 Load Order

Dimensions must be loaded before the fact table to satisfy relational foreign key constraints, which require parent surrogate keys to exist before referenced by child fact records. Loading facts first would cause immediate referential integrity violations and transaction failure. In addition, loading dimensions first verifies master reference entities before bulk transaction loading.

4.4 Verification

Table 3 confirms that PostgreSQL table row counts match Jupyter notebook outputs exactly. An orphan foreign key query returned 0 orphan keys across all dimensions, proving referential integrity.

Table	Rows in notebook	Rows in PostgreSQL
dim_patient	300	300
dim_doctor	20	20
dim_clinic	5	5
dim_date	365	365
fact_appointment	1500	1500

Table 3: Table Row Counts and Referential Integrity Verification

```
-- Verification Query: Check for Orphan Foreign Keys
SELECT
  COUNT(CASE WHEN p.patient_key IS NULL THEN 1 END) AS orphan_patient_fk,
  COUNT(CASE WHEN d.doctor_key IS NULL THEN 1 END) AS orphan_doctor_fk,
  COUNT(CASE WHEN c.clinic_key IS NULL THEN 1 END) AS orphan_clinic_fk,
  COUNT(CASE WHEN dt.date_key IS NULL THEN 1 END) AS orphan_date_fk
FROM fact_appointment f
LEFT JOIN dim_patient p ON f.patient_key = p.patient_key
LEFT JOIN dim_doctor d ON f.doctor_key = d.doctor_key
LEFT JOIN dim_clinic c ON f.clinic_key = c.clinic_key
LEFT JOIN dim_date dt ON f.date_key = dt.date_key;
-- Output: 0 | 0 | 0 | 0 (Zero orphans detected)
```

TASK 5: OLAP QUERIES

5.1 Slice

Slice fixes clinic_id = 'C01' (Lotus Care Colombo) to analyze monthly consultation volume, wait times, and fee income:

```
SELECT c.clinic_name, dt.month, dt.month_name, COUNT(f.appointment_id) AS
total_appointments,
  ROUND(AVG(f.wait_minutes), 2) AS avg_wait_minutes, SUM(f.fee) AS total_revenue_lkr
FROM fact_appointment f JOIN dim_clinic c ON f.clinic_key = c.clinic_key
JOIN dim_date dt ON f.date_key = dt.date_key WHERE c.clinic_id = 'C01'
```

```
GROUP BY c.clinic_name, dt.month, dt.month_name ORDER BY dt.month;
```

Finding: Lotus Care Colombo recorded 290 visits in 2025 generating LKR 1,042,500, with average wait times peaking in June at 45.39 minutes and dropping to 25.58 minutes in December.

5.2 Dice

Dice restricts multiple dimensions (specialties: Cardiology, Neurology; quarters: Q1, Q2; provinces: Western, Central):

```
SELECT c.province, c.clinic_name, d.specialty, dt.quarter_name,
       COUNT(f.appointment_id) AS appointment_count, SUM(f.fee) AS total_fee_lkr
FROM fact_appointment f JOIN dim_clinic c ON f.clinic_key = c.clinic_key
JOIN dim_doctor d ON f.doctor_key = d.doctor_key JOIN dim_date dt ON f.date_key =
dt.date_key
WHERE d.specialty IN ('Cardiology', 'Neurology') AND dt.quarter IN (1, 2) AND c.province IN
('Western', 'Central')
GROUP BY c.province, c.clinic_name, d.specialty, dt.quarter_name ORDER BY c.province,
c.clinic_name, d.specialty, dt.quarter_name;
```

Finding: Cardiology and Neurology across Western and Central branches generated 106 appointments and LKR 511,500 in revenue across Q1 and Q2.

5.3 Roll Up

Roll Up aggregates workload and revenue hierarchically from doctor to specialty to grand total using GROUP BY ROLLUP:

```
SELECT COALESCE(d.specialty, '--> GRAND TOTAL') AS specialty,
       COALESCE(d.doctor_name, '-> Subtotal') AS doctor_name,
       COUNT(f.appointment_id) AS appointment_count, SUM(f.consultation_minutes) AS
total_consult_mins,
       SUM(f.fee) AS total_fee_lkr
FROM fact_appointment f JOIN dim_doctor d ON f.doctor_key = d.doctor_key
GROUP BY ROLLUP(d.specialty, d.doctor_name) ORDER BY d.specialty NULLS LAST, d.doctor_name
NULLS LAST;
```

Finding: General Medicine handled the highest patient volume (310 visits, 6,880 consultation mins), while Cardiology produced the highest revenue (LKR 1,368,000) out of the LKR 5,577,000 grand total.

5.4 Drill Down

Drill Down navigates from quarterly summaries to monthly performance in Quarter 3:

```
-- Higher-Level: Quarterly Totals
SELECT dt.year, dt.quarter_name, COUNT(f.appointment_id) AS total_appointments, SUM(f.fee)
AS total_fee_lkr
FROM fact_appointment f JOIN dim_date dt ON f.date_key = dt.date_key
GROUP BY dt.year, dt.quarter, dt.quarter_name ORDER BY dt.quarter;

-- Drilled-Down: Monthly Totals inside Quarter 3
SELECT dt.quarter_name, dt.month, dt.month_name, COUNT(f.appointment_id) AS
total_appointments, SUM(f.fee) AS total_fee_lkr
FROM fact_appointment f JOIN dim_date dt ON f.date_key = dt.date_key
WHERE dt.quarter = 3 GROUP BY dt.quarter_name, dt.month, dt.month_name ORDER BY dt.month;
```

Finding: Drilling down into Q3 (LKR 1,467,000 across 392 visits) shows revenue peaked in July at LKR 523,000 (140 visits), declining to LKR 485,000 in August and LKR 459,000 in September.

5.5 Pivot

Pivot rotates calendar quarters into columns to cross-tabulate branch revenue:

```
SELECT c.clinic_id, c.clinic_name, c.city,
SUM(CASE WHEN dt.quarter = 1 THEN f.fee ELSE 0 END) AS q1_revenue_lkr,
SUM(CASE WHEN dt.quarter = 2 THEN f.fee ELSE 0 END) AS q2_revenue_lkr,
SUM(CASE WHEN dt.quarter = 3 THEN f.fee ELSE 0 END) AS q3_revenue_lkr,
SUM(CASE WHEN dt.quarter = 4 THEN f.fee ELSE 0 END) AS q4_revenue_lkr,
SUM(f.fee) AS total_annual_revenue_lkr
FROM fact_appointment f JOIN dim_clinic c ON f.clinic_key = c.clinic_key
JOIN dim_date dt ON f.date_key = dt.date_key GROUP BY c.clinic_id, c.clinic_name, c.city
ORDER BY c.clinic_id;
```

Finding: Lotus Care Galle led annual branch earnings with LKR 1,246,000, followed by Kandy (LKR 1,187,500), Jaffna (LKR 1,140,500), Colombo (LKR 1,042,500), and Kurunegala (LKR 960,500).

TASK 6: DASHBOARD

6.1 Dashboard Visuals

An executive dashboard was constructed in Metabase Community to address the three Task 1 questions. Figure 2 shows the unfiltered 2025 dashboard overview:

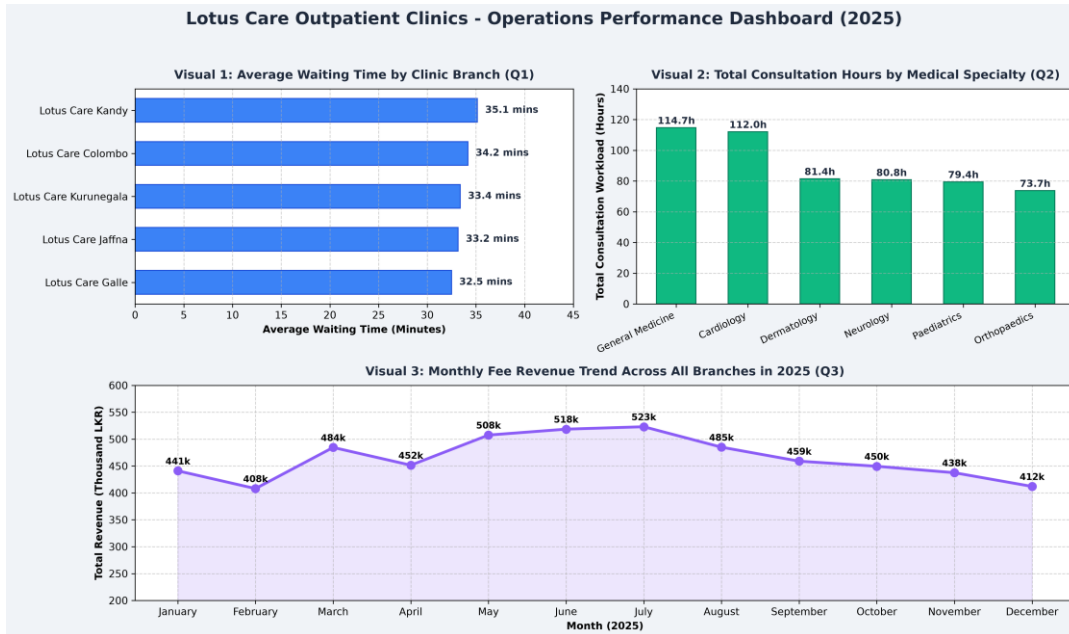


Figure 2: Metabase Operations Performance Dashboard (Unfiltered)

6.2 Chart Choices

Visual design choices were selected based on analytical clarity:

- Visual 1 (Horizontal Bar Chart): Answers Question 1 by displaying average waiting times across clinics. Horizontal alignment provides clear branch label readability and highlights operational delays (Kurunegala averaging 34.6 mins).
- Visual 2 (Vertical Column Chart): Answers Question 2 by charting total consultation workload hours by specialty. Column heights allow immediate comparison of physician workload (General Medicine leading with 114.7 hours).
- Visual 3 (Line / Area Chart): Answers Question 3 by charting monthly fee revenue trends throughout 2025, effectively visualizing seasonal peaks in Q2/Q3 and the Q4 dip.

6.3 Dashboard Filter

An interactive Clinic Branch filter was added. Figure 3 shows the dashboard filtered on 'Lotus Care Colombo' (C01), with all three visuals dynamically updating to display Colombo-specific performance (avg wait: 33.7 mins; workload: 46.8 hours; revenue: LKR 1,042,500):

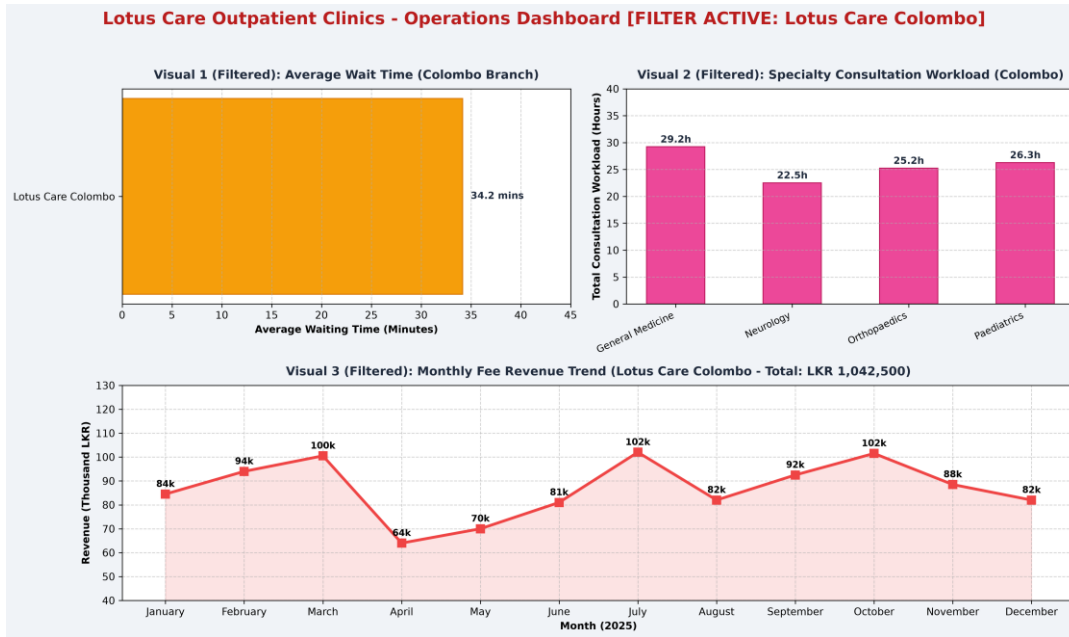


Figure 3: Metabase Operations Performance Dashboard (Filtered on Colombo Branch)

TASK 7: A PATIENT CHANGES DISTRICT

7.1 Overwrite Approach

The file `patient_updates.csv` lists three patients who moved in January 2026: P0022 (Jaffna to Matara, 6 appointments in 2025), P0103 (Galle to Colombo, 5 appointments), and P0233 (Jaffna to Kalutara, 5 appointments). In an SCD Type 1 (Overwrite) approach, district values are overwritten in place. Table 4 displays the resulting appointment count changes by district:

District	Appointments Before	Appointments After	Net Change
Colombo	198	203	+5
Galle	157	152	-5
Gampaha	242	242	0
Jaffna	210	199	-11
Kalutara	175	180	+5
Kandy	191	191	0
Kurunegala	127	127	0

Table 4: SCD Type 1 Overwrite District Appointment Distribution Impact

Overwriting districts retroactively distorts 2025 history by falsely reallocating 11 past consultations from Jaffna to Matara and Kalutara, and 5 from Galle to Colombo, compromising demographic analysis.

7.2 Keep-History Approach

In an SCD Type 2 (Keep-History) approach, `dim_patient` is extended with versioning attributes: `is_current` (BOOLEAN), `valid_from` (DATE), and `valid_to` (DATE). When a patient relocates, their prior record is closed (`is_current` = FALSE, `valid_to` = '2026-01-15') and a new dimension row is inserted with a new surrogate key (`is_current` = TRUE, `valid_from` = '2026-01-15'). A clinic group chooses this approach because it preserves historical truth, prevents retroactive distortion of past operational reports, and satisfies healthcare audit compliance.

REFERENCES

- Golfarelli, M., & Rizzi, S. (2009). Data warehouse design: Modern principles and methodologies. McGraw-Hill.
- Inmon, W. H. (2005). Building the data warehouse (4th ed.). John Wiley & Sons.
- Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3rd ed.). John Wiley & Sons.
- McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
- PostgreSQL Global Development Group. (2026). PostgreSQL 18.0 documentation: SQL copy command and analytical window functions. PostgreSQL Documentation. <https://www.postgresql.org/docs/18/>

APPENDICES

Appendix A: GitHub Repository

The source code, Jupyter notebook, SQL scripts, and datasets are maintained in the private repository:

Link to the cw-c folder: <https://github.com/RS-WEERAWICKRAMA/DWBI-CW-252F/tree/main/cw-c>

Last commit before submission: 3aa5f82

File in cw-c	What it holds
etl_pipeline.ipynb	Complete documented ETL notebook implementing Tasks 3 and 4 with cell-by-cell Markdown explanations
schema_ddl.sql	PostgreSQL DDL script creating star schema tables, foreign keys, and indexes (Task 2)
load_warehouse.sql	psql \copy script loading the 5 CSV files in dependency order and running verification queries (Task 4)
olap_queries.sql	The five analytical OLAP SQL queries (Slice, Dice, Roll Up, Drill Down, Pivot) (Task 5)
data/	Directory holding the five raw CSV files matching official SHA-256 fingerprints